SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	Akash J	**Reg. No.:**	192324295
Section / Batch:	2023	**Date:**	22-10-2026

Code of Conduct

*I **Akash J / 192324295** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **Akash J**

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Annexure A – Coding Challenge

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. **Problem Statement:**

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Develop a Python application that validates:
Email Address
Password
Mobile Number

using Regular Expressions.

Validation Rules
Email
Must start with an alphabet.
May contain letters, digits, '.', '_' before '@'.
Domain should contain only alphabets.
Valid extensions: .com, .org, .edu, .net, .in
Password
Minimum 8 characters
At least one uppercase letter
At least one lowercase letter
At least one digit
At least one special character (@,#,$,%,&,!)
Mobile Number
Must contain exactly 10 digits.
First digit should be between 6–9.
Expected Output

Display:

Valid Email / Invalid Email
Strong Password / Weak Password
Valid Mobile Number / Invalid Mobile Number

Section 2: Design a Finite State Automata (FSA) Simulator

2. **Problem Statement**

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- Accept the DFA description
    - States
    - Input alphabet
    - Transition table
    - Initial state
    - Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

Example

Language:
Strings ending with "ab"

Input
abaab

Output
Transition Path:
q0 → q1 → q0 → q1 → q2
Accepted

Section 3: Smart Pattern Matching Engine using Regular Expressions

3. Problem Statement

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:
Word Search
Prefix Search
Suffix Search
Date Search
Phone Number Search
Hashtag Search
Mention (@username) Search

Example Input
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing

User Menu
1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix

Expected Output
Display all matching patterns based on user selection.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Section 1: Intelligent Email and Password Validator

Problem

Validate an email address, a password and a mobile number using regular expressions before a user account is created.

Regular expressions used

```
Email    : ^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$
Password : ^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!]).{8,}$
Mobile   : ^[6-9][0-9]{9}$
```

Program (q1_validator.py)

```
import re
import sys

EMAIL_PATTERN = r"^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$"
PASSWORD_PATTERN = r"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!]).{8,}$"
MOBILE_PATTERN = r"^[6-9][0-9]{9}$"


def validate_email(email):
    return re.fullmatch(EMAIL_PATTERN, email) is not None


def validate_password(password):
    return re.fullmatch(PASSWORD_PATTERN, password) is not None


def validate_mobile(mobile):
    return re.fullmatch(MOBILE_PATTERN, mobile) is not None


def ask(prompt):
    value = input(prompt)
    if not sys.stdin.isatty():      # echo typed text when input is piped
        print(value)
    return value


def main():
    print("=" * 50)
    print(" USER REGISTRATION - CREDENTIAL VALIDATOR")
    print("=" * 50)
    while True:
        email = ask("\nEnter Email    : ").strip()
        password = ask("Enter Password : ").strip()
        mobile = ask("Enter Mobile   : ").strip()

        print("\n--- Result ---")
        print("Valid Email" if validate_email(email) else "Invalid Email")
        print("Strong Password" if validate_password(password) else "Weak Password")
        print("Valid Mobile Number" if validate_mobile(mobile) else "Invalid Mobile Number")

        again = ask("\nValidate another? (y/n): ").strip().lower()
        if again != "y":
            print("Thank you!")
            break


if __name__ == "__main__":
    main()
```

Output – Section 1

```
python q1_validator.py

================================================
 USER REGISTRATION - CREDENTIAL VALIDATOR
================================================

Enter Email    : john.doe_21@gmail.com
Enter Password : Strong@123
Enter Mobile   : 9876543210

--- Result ---
Valid Email
Strong Password
Valid Mobile Number

Validate another? (y/n): y

Enter Email    : 1ohn@gmail.com
Enter Password : weakpass
Enter Mobile   : 5876543210

--- Result ---
Invalid Email
Weak Password
Invalid Mobile Number

Validate another? (y/n): y

Enter Email    : alice@yahoo.xyz
Enter Password : Pass@word1
Enter Mobile   : 98765

--- Result ---
Invalid Email
Strong Password
Invalid Mobile Number

Validate another? (y/n): n
Thank you!
```

Section 2: Finite State Automata (DFA) Simulator

Problem

Build a DFA simulator that accepts the states, input alphabet, transition table, initial state and final states, then tests multiple input strings, prints the transition path and reports Accepted / Rejected.
Language used for the demo: strings ending with "ab" (alphabet {a, b}, start q0, final q2).

Program (q2_dfa.py)

```
import sys


def ask(prompt):
    value = input(prompt)
    if not sys.stdin.isatty():      # echo typed text when input is piped
        print(value)
    return value


class DFA:
    def __init__(self, states, alphabet, transitions, start, finals):
        self.states = states
        self.alphabet = alphabet
        self.transitions = transitions      # {(state, symbol): next_state}
        self.start = start
        self.finals = finals

    def simulate(self, string):
        """Return (path, accepted)."""
        current = self.start
        path = [current]
        for ch in string:
            if ch not in self.alphabet:
                return path, False, f"Symbol '{ch}' is not in the alphabet"
            nxt = self.transitions.get((current, ch))
            if nxt is None:
                return path, False, f"No transition from {current} on '{ch}'"
            current = nxt
            path.append(current)
        return path, current in self.finals, ""


def read_dfa():
    print("=" * 50)
    print(" DFA SIMULATOR")
    print("=" * 50)
    states = [s.strip() for s in ask("Enter states (comma separated)    : ").split(",")]
    alphabet = [s.strip() for s in ask("Enter input alphabet (comma sep.) : ").split(",")]
    start = ask("Enter initial state               : ").strip()
    finals = [s.strip() for s in ask("Enter final state(s) (comma sep.) : ").split(",")]

    print("\nEnter transition table row by row (one state at a time)")
    transitions = {}
    for st in states:
        for sym in alphabet:
            nxt = ask(f"  delta({st}, {sym}) = ").strip()
            transitions[(st, sym)] = nxt
    return DFA(states, alphabet, transitions, start, finals)

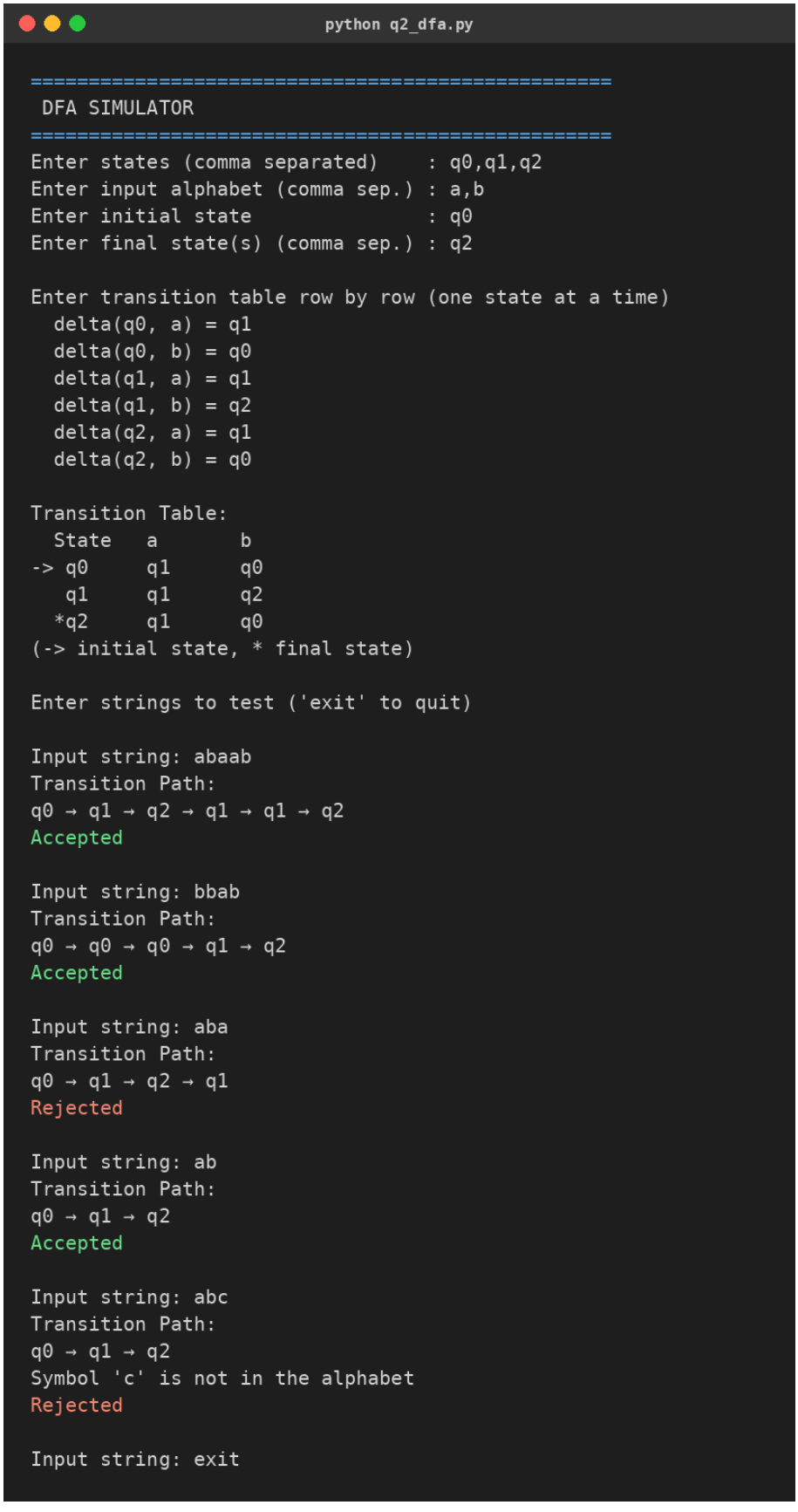

def show_table(dfa):
    print("\nTransition Table:")
```

```
    print("  State".ljust(10) + "".join(s.ljust(8) for s in dfa.alphabet))
    for st in dfa.states:
        tag = ("->" if st == dfa.start else "  ") + ("*" if st in dfa.finals else " ")
        print((tag + st).ljust(10) + "".join(dfa.transitions[(st, a)].ljust(8) for a in dfa.alphabet))
    print("(-> initial state, * final state)")


def main():
    dfa = read_dfa()
    show_table(dfa)
    print("\nEnter strings to test ('exit' to quit)")
    while True:
        s = ask("\nInput string: ").strip()
        if s.lower() == "exit":
            break
        path, ok, reason = dfa.simulate(s)
        print("Transition Path:")
        print(" → ".join(path))
        if reason:
            print(reason)
        print("Accepted" if ok else "Rejected")


if __name__ == "__main__":
    main()
```

Output – Section 2



Section 3: Smart Pattern Matching Engine

Problem

Build a text search engine using regular expressions supporting word, prefix, suffix, date, phone number, hashtag and mention search through a menu.

Program (q3_search.py)

```
import re
import sys

TEXT = """Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing"""


def ask(prompt):
    value = input(prompt)
    if not sys.stdin.isatty():      # echo typed text when input is piped
        print(value)
    return value


def search_word(text, word):
    return re.findall(rf"\b{re.escape(word)}\b", text, flags=re.IGNORECASE)


def search_prefix(text, prefix):
    return re.findall(rf"\b{re.escape(prefix)}\w*", text, flags=re.IGNORECASE)


def search_suffix(text, suffix):
    return re.findall(rf"\b\w*{re.escape(suffix)}\b", text, flags=re.IGNORECASE)


def search_date(text):
    return re.findall(r"\b\d{1,2}[/-]\d{1,2}[/-]\d{4}\b", text)


def search_phone(text):
    return re.findall(r"(?<!\d)[6-9]\d{9}(?!\d)", text)


def search_hashtag(text):
    return re.findall(r"#\w+", text)


def search_mention(text):
    return re.findall(r"@\w+", text)


def show(title, results):
    print(f"\n{title}:")
    if results:
        for i, r in enumerate(results, 1):
            print(f"  {i}. {r}")
    else:
        print("  No match found")


def main():
    print("=" * 50)
```

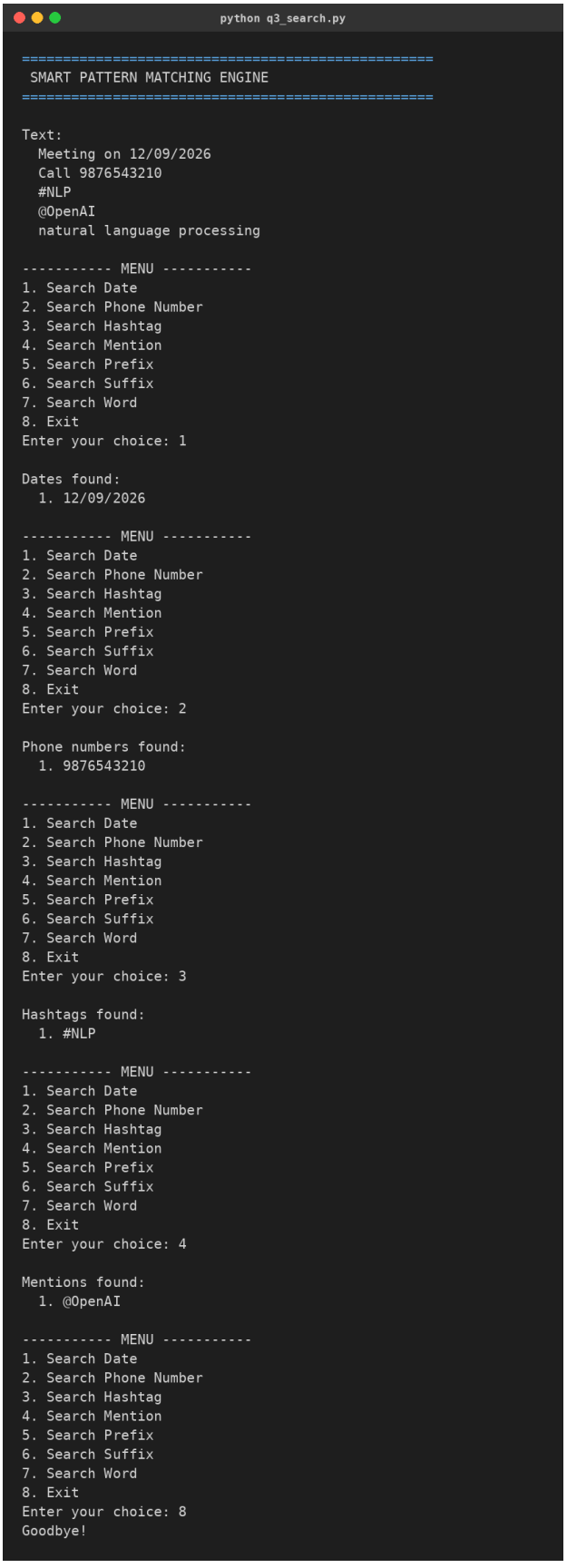
```
    print(" SMART PATTERN MATCHING ENGINE")
    print("=" * 50)
    print("\nText:")
    for line in TEXT.splitlines():
        print("  " + line)

    while True:
        print("\n----------- MENU -----------")
        print("1. Search Date")
        print("2. Search Phone Number")
        print("3. Search Hashtag")
        print("4. Search Mention")
        print("5. Search Prefix")
        print("6. Search Suffix")
        print("7. Search Word")
        print("8. Exit")
        choice = ask("Enter your choice: ").strip()

        if choice == "1":
            show("Dates found", search_date(TEXT))
        elif choice == "2":
            show("Phone numbers found", search_phone(TEXT))
        elif choice == "3":
            show("Hashtags found", search_hashtag(TEXT))
        elif choice == "4":
            show("Mentions found", search_mention(TEXT))
        elif choice == "5":
            p = ask("Enter prefix: ").strip()
            show(f"Words starting with '{p}'", search_prefix(TEXT, p))
        elif choice == "6":
            s = ask("Enter suffix: ").strip()
            show(f"Words ending with '{s}'", search_suffix(TEXT, s))
        elif choice == "7":
            w = ask("Enter word: ").strip()
            show(f"Matches for '{w}'", search_word(TEXT, w))
        elif choice == "8":
            print("Goodbye!")
            break
        else:
            print("Invalid choice, try again.")


if __name__ == "__main__":
    main()
```

Output – Section 3 (Date, Phone, Hashtag, Mention)



Output – Section 3 (Prefix, Suffix, Word)

